

哈尔滨工业大学 2026 春

22CS22013 数据库系统实验报告

姓名	朱梓萌	学号	2023111396
专业	人工智能	班级	23R0362

实验报告评价（仅由教师填写）

评语	
得分	

实验 1：缓冲区管理器实现

一、实验目标

本实验的目标是：

1. 掌握 Rucbase 缓冲池页面替换策略的实现方法。
2. 掌握 Rucbase 缓冲池管理器的实现方法。

二、实验内容

实验 1 包含两个核心任务：

1. 补全 LRURemplacer，实现最近最少使用页替换策略。
2. 补全 BufferPoolManager，实现页面获取、取消固定、刷盘、分配新页、删除页等功能。

在整体架构上，实验 1 要解决的是“数据库如何管理内存中的页”。

Rucbase 中磁盘上的页不会被每次直接读写，而是先进入缓冲池；缓冲池空间有限，因此需要：

1. 判断一个页是否已经在内存中。
2. 缓冲池满时决定淘汰哪个页。
3. 被淘汰页若是脏页，要先写回磁盘。
4. 正在被使用的页不能被淘汰。

因此，LRURemplacer 和 BufferPoolManager 分别解决两个层次的问题：

1. LRURemplacer 负责“如果必须替换，应该替换谁”。
2. BufferPoolManager 负责“什么时候替换、如何替换、如何和磁盘交互”。

三、主要实现代码与实现说明

1. LRURemplacer 的实现

LRURemplacer 是缓冲池中的页面替换器。

它维护“当前哪些 frame 可以被淘汰”以及“这些 frame 的最近访问顺序”。

(1) 核心数据结构说明

在阅读 lru_replacer.h 时，可以把几个成员理解为：

1. LRUList_
保存所有可淘汰 frame 的访问顺序。链表头表示最近使用，链表尾表示最久未使用。
2. LRUhash_
保存 frame_id -> 链表迭代器 的映射，用于快速判断和快速删除。
3. latch_
用于并发控制，防止多个线程同时修改内部结构。
4. max_size_
表示替换器最多能维护多少个 frame，通常与缓冲池容量一致。

因此，这个类的本质是“链表维护顺序，哈希表维护定位”的标准 LRU 结构。

(2) LRURemplacer::victim

```
bool LRURemplacer::victim(frame_id_t* frame_id) {  
    std::scoped_lock lock{latch_};  
    if (LRUList_.empty()) {  
        frame_id = nullptr;  
    }
```

```

        return false;
    }
    *frame_id = LRUlist_.back();
    LRUlist_.pop_back();
    LRUhash_.erase(*frame_id);
    return true;
}

```

实现说明：

1. 首先对内部状态加锁，防止并发环境下链表和哈希表被同时修改。
2. 若 LRUlist_ 为空，说明当前没有任何可淘汰 frame，函数直接返回 false。
3. 若链表非空，则从链表尾部取出一个 frame。之所以取尾部，是因为尾部表示“最久未使用”的页框。
4. 取出后需要同步更新两个数据结构：
 - 从 LRUlist_ 中删除该 frame。
 - 从 LRUhash_ 中删除该 frame 的迭代器映射。

这一函数在语义上回答了“当缓冲池必须换页时，该淘汰谁”的问题。

(3) LRUREplacer::pin

```

void LRUREplacer::pin(frame_id_t frame_id) {
    std::scoped_lock lock{latch_};
    if (LRUhash_.find(frame_id) == LRUhash_.end()) return;

    auto item = LRUhash_[frame_id];
    LRUlist_.erase(item);
    LRUhash_.erase(frame_id);
}

```

实现说明：

1. pin 的含义是：该页正在被使用，因此当前不能被替换。
2. 在缓冲池中，一个页只要 pin_count_ > 0，就不应该进入可淘汰集合。
3. 因此，如果这个 frame 已经在 LRU 队列中，就必须把它删除。
4. 删除时依然通过哈希表快速定位链表节点，从而实现 O(1) 删除。

这个函数体现的是“只要页还被引用，就不能参与替换”的原则。

(4) LRUREplacer::unpin

```

void LRUREplacer::unpin(frame_id_t frame_id) {
    std::scoped_lock lock{latch_};
    if (LRUhash_.find(frame_id) != LRUhash_.end()) {
        return;
    } else {
        LRUlist_.push_front(frame_id);
        LRUhash_[frame_id] = LRUlist_.begin();
    }

    if (Size() > max_size_) {
        auto item = LRUlist_.back();
        LRUhash_.erase(item);
    }
}

```

```

        LRUList_.pop_back();
    }
}

```

实现说明：

1. unpin 表示页已经不再被当前执行流使用，因此可以重新成为候选淘汰页。
2. 若该 frame 已经在 LRU 队列中，则说明它此前已经被登记为可淘汰，无需重复插入。
3. 若不在 LRU 队列中，则把它插入链表头部。这样做的含义是：它是最近刚刚变成可淘汰状态的，因此在 LRU 意义上“更新”。
4. 同时在哈希表中记录它对应的链表位置。
5. 若当前队列长度超过最大容量，则从尾部删除最老元素，并同步删除哈希表中的项。

这里需要注意，unpin 并不是“把页内容写回磁盘”，而只是改变它在替换器中的状态。

2. BufferPoolManager 的实现

如果说 LRURemplacer 解决的是“替换谁”，那么 BufferPoolManager 解决的是“如何管理一整个缓冲池”。

(1) 核心数据结构说明

阅读 buffer_pool_manager.h 时，可以将几个关键成员理解为：

1. pages_
缓冲池中真实存放页内容的数组，每个元素对应一个 frame。
2. page_table_
页表，维护 PageId->frame_id 的映射，用于判断一个逻辑页是否已经在缓冲池中。
3. free_list_
保存尚未被使用的空闲 frame。
4. replacer_
指向页替换器，这里具体实现为 LRU。
5. disk_manager_
负责和磁盘交互，完成实际的读页、写页、分配页号、释放页号等操作。
6. latch_
用于保护缓冲池管理器的共享结构，防止多个线程同时修改页表、空闲链表等。

整个缓冲池的典型工作流程是：

1. 用户请求一个页面。
2. 先查页表，看它是否已经在内存。
3. 若命中，直接返回。
4. 若未命中，则先找空闲 frame；若没有空闲 frame，则找 victim。
5. victim 若为脏页，则先写盘。
6. 再把新页读入该 frame，并更新页表。

(2) BufferPoolManager::find_victim_page

```

bool BufferPoolManager::find_victim_page(frame_id_t* frame_id) {
    if (!free_list_.empty()) {
        *frame_id = free_list_.front();
        free_list_.pop_front();
        return true;
    }
    return replacer_>victim(frame_id);
}

```

```
}
```

实现说明：

该函数用于寻找一个“可以放新页的 frame”。

整体思路分成两种情况：

1. 若缓冲池中仍有从未使用过的空闲 frame，则优先从 free_list_ 取出一个。
2. 若空闲 frame 已全部用完，则只能通过替换器从已有页中挑选一个 victim。

这种设计符合数据库缓冲池的常规策略：

优先使用新空间，只在必要时才替换已有页。

(3) BufferPoolManager::fetch_page

```
Page* BufferPoolManager::fetch_page(PageId page_id) {
    std::scoped_lock lock{latch_};
    Page* target_page;
    frame_id_t target_frame_id;
    if (page_table_.find(page_id) != page_table_.end()) {
        target_frame_id = page_table_[page_id];
        target_page = &pages_[target_frame_id];
        target_page->pin_count_++;
        replacer_->pin(target_frame_id);
    } else {
        if (!find_victim_page(&target_frame_id)) return nullptr;
        target_page = &pages_[target_frame_id];

        if (target_page->is_dirty()) {
            PageId id = target_page->get_page_id();
            disk_manager_->write_page(id.fd, id.page_no, target_page->data_, PAGE_SIZE);
            target_page->is_dirty_ = false;
        }
        target_page->reset_memory();
        auto item = page_table_.find(target_page->id_);
        if (item != page_table_.end()) page_table_.erase(item);
        page_table_[page_id] = target_frame_id;
        target_page->id_ = page_id;

        if (target_page->id_.page_no != INVALID_PAGE_ID) {
            disk_manager_->read_page(page_id.fd, page_id.page_no, target_page->data_,
PAGE_SIZE);
        }
        target_page->pin_count_ = 1;
        replacer_->pin(target_frame_id);
    }
    return target_page;
}
```

实现说明：

这是缓冲池管理器最关键的函数，它承担“读页并放入缓冲池”的职责。

可以把它分解成两类情况：

1. 页已经在缓冲池中。
 - 通过 `page_table_` 找到对应的 `frame_id`。
 - 增加该页的 `pin_count_`。
 - 调用 `replacer_>pin()` 将它从候选淘汰集合中移除。
 - 直接返回页指针。
2. 页不在缓冲池中。
 - 先调用 `find_victim_page()` 找到一个可用 `frame`。
 - 若找不到可用 `frame`，则返回 `nullptr`。
 - 若旧页是脏页，需要先调用 `write_page()` 写回磁盘。
 - 清空旧页内存，并删除旧页在页表中的映射。
 - 将新的 `page_id` 映射到当前 `frame`。
 - 若该页是有效逻辑页，则从磁盘将其内容读入内存。
 - 设置 `pin_count_ = 1`，表示该页当前正被使用。
 - 再通过 `replacer_>pin()` 将其移出候选淘汰集合。

从实现角度看，`fetch_page()` 同时处理了：

1. 缓冲池命中。
2. 缓冲池缺页。
3. `victim` 脏页写回。
4. 页表更新。
5. 新页装入。

因此这是实验 1 中最核心的流程函数。

(4) `BufferPoolManager::unpin_page`

```
bool BufferPoolManager::unpin_page(PageId page_id, bool is_dirty) {
    std::scoped_lock lock{latch_};
    auto item = page_table_.find(page_id);
    if (item == page_table_.end()) {
        return false;
    }
    frame_id_t frame_id = page_table_[page_id];
    Page* target_page = &pages_[frame_id];

    if (target_page->pin_count_ == 0) return false;
    target_page->pin_count_--;

    if (target_page->pin_count_ == 0) replacer_>unpin(frame_id);
    target_page->is_dirty_ = is_dirty;
    return true;
}
```

实现说明：

当某个调用者不再使用页面时，需要调用 `unpin_page()`。

该函数的执行逻辑如下：

1. 先从页表中找到该页所在 `frame`。
2. 如果页表中不存在该页，说明缓冲池里没有这页，返回 `false`。

3. 如果 `pin_count_` 已经是 0，再次 `unpin` 属于非法情况，也返回 `false`。
4. 否则，将 `pin_count_` 减一。
5. 如果减到 0，说明该页当前不再被任何执行流使用，这时调用 `replacer_>unpin(frame_id)`，把它放回可淘汰集合。
6. 最后根据参数更新脏页标记。

也就是说，`unpin_page()` 主要完成两件事：

1. 更新引用计数。
2. 在恰当时机把页重新交还给替换器。

(5) `BufferPoolManager::flush_page`

```
bool BufferPoolManager::flush_page(PageId page_id) {
    std::scoped_lock lock{latch_};
    auto item = page_table_.find(page_id);
    if (item == page_table_.end()) return false;

    frame_id_t frame_id = page_table_[page_id];
    Page* target_page = &pages_[frame_id];
    disk_manager_>write_page(target_page->get_page_id().fd,
                             target_page->get_page_id().page_no,
                             target_page->get_data(),
                             PAGE_SIZE);

    target_page->is_dirty_ = false;
    return true;
}
```

实现说明：

此函数的作用是强制将指定页写回磁盘。

与 `victim` 替换时“必要时才写脏页”不同，`flush_page()` 表示显式刷盘请求，因此只要页存在于缓冲池中，就无条件写盘，并将 `is_dirty_` 清零。

(6) `BufferPoolManager::new_page`

```
Page* BufferPoolManager::new_page(PageId* page_id) {
    std::scoped_lock lock{latch_};
    frame_id_t frame_id;
    if (find_victim_page(&frame_id)) {
        page_id->page_no = disk_manager_>allocate_page(page_id->fd);
        Page *page = &pages_[frame_id];

        if (page->is_dirty()) {
            PageId id = page->get_page_id();
            disk_manager_>write_page(id.fd, id.page_no, page->data_, PAGE_SIZE);
            page->is_dirty_ = false;
        }

        page->reset_memory();
        auto item = page_table_.find(page->id_);
        if (item != page_table_.end()) page_table_.erase(item);
    }
}
```

```

        page_table_[*page_id] = frame_id;
        page->id_ = *page_id;

        replacer->pin(frame_id);
        page->pin_count_ = 1;
        return page;
    }
    return nullptr;
}

```

实现说明：

`new_page()` 的功能是“在磁盘上为指定文件分配一个新的逻辑页，并在缓冲池中为它找到一个 `frame`”。

它的步骤可以理解为：

1. 先找一个可用 `frame`。
2. 调用 `disk_manager->allocate_page()` 为该文件申请新的页号。
3. 若 `frame` 中原有页是脏页，先写回磁盘。
4. 清空内存，并删除旧页映射。
5. 建立新页号到当前 `frame` 的映射关系。
6. 将新页标记为固定，`pin_count_ = 1`。

与 `fetch_page()` 的区别在于：

1. `fetch_page()` 是读取已有页。
2. `new_page()` 是创建新页，因此不需要从磁盘中再读数据。

(7) `BufferPoolManager::delete_page`

```

bool BufferPoolManager::delete_page(PageId page_id) {
    std::scoped_lock lock{latch_};
    auto item = page_table_.find(page_id);
    if (item == page_table_.end()) return true;

    frame_id_t frame_id = page_table_[page_id];
    Page* target_page = &pages_[frame_id];
    if (target_page->pin_count_ != 0) return false;

    disk_manager_->deallocate_page(target_page->get_page_id().page_no);

    if (target_page->is_dirty()) {
        PageId id = target_page->get_page_id();
        disk_manager_->write_page(id.fd, id.page_no, target_page->data_, PAGE_SIZE);
        target_page->is_dirty_ = false;
    }
    target_page->reset_memory();
    target_page->is_dirty_ = false;
    target_page->pin_count_ = 0;

    auto item_ = page_table_.find(target_page->id_);
}

```



```

        if (item_ != page_table_.end()) page_table_.erase(item_);

        free_list_.push_back(frame_id);
        return true;
    }
}
实现说明：
删除页时需要特别注意“页是否仍在使用”：
1. 若目标页不在缓冲池中，直接认为删除成功。
2. 若该页仍被使用，即 pin_count_ != 0，则不能删除，返回 false。
3. 若可以删除，则释放其磁盘页号。
4. 若该页为脏页，仍需要先写盘，避免内容丢失。
5. 清空页内容和元信息。
6. 删除页表项。
7. 最后把 frame 放回空闲链表中。
(8) BufferPoolManager::flush_all_pages
void BufferPoolManager::flush_all_pages(int fd) {
    for (int i = 0; i < pool_size_; i++) {
        Page* target_page = &pages_[i];
        if (target_page->get_page_id().page_no != INVALID_PAGE_ID
            && target_page->get_page_id().fd == fd) {
            disk_manager_>write_page(fd,
                                     target_page->get_page_id().page_no,
                                     target_page->get_data(),
                                     PAGE_SIZE);
            target_page->is_dirty_ = false;
        }
    }
}
}

```

实现说明：

该函数遍历整个缓冲池，将属于某个文件 fd 的全部页统一刷回磁盘。
这通常用于关闭数据库或关闭文件时的“整体落盘”操作。

四、实验问题回答

1. LRUList_的作用是什么？

LRUList_ 是一个双向链表，保存所有当前可淘汰的 frame，并按照最近访问顺序排列。链表头部表示最近使用，尾部表示最久未使用，因此尾部 frame 可以作为 victim。

2. LRUhash_的作用是什么？

LRUhash_ 维护 frame_id 到链表迭代器的映射，用于快速判断某个 frame 是否在 LRU 队列中，并支持 O(1) 时间复杂度的删除和插入。

3. LRUList_和 LRUhash_的关系是什么？

LRUList_ 负责维护访问顺序，LRUhash_ 负责快速定位节点。二者结合后，可以同时兼顾“顺序管理”和“高效操作”。

4. Page::is_dirty_的作用是什么？

is_dirty_ 表示页面是否被修改但尚未写回磁盘。若为真，页面在替换或刷盘时必须写回磁盘，

以保证数据一致性。

5. Page::pin_count_ 的作用是什么？

pin_count_ 表示当前有多少执行流正在使用该页。只要 pin_count_ > 0, 该页就不能被替换。

6. BufferPoolManager::page_table_ 的作用是什么？

页表维护逻辑页号到内存 frame 的映射，使缓冲池可以快速判断一个页是否命中以及它所在的 frame。

7. BufferPoolManager::free_list_ 的作用是什么？

free_list_ 保存所有尚未使用的空闲 frame，缓冲池会优先使用空闲 frame，而不是直接替换已有页。

五、测试结果

```
amdreameng@LAPTOP-34HBN43K:/mnt/d/Desktop/数据库系统/lab/rucbase-lab/build$ ./bin/buffer_pool_manager_test
Running main() from /mnt/d/Desktop/数据库系统/lab/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from BufferPoolManagerTest
[ RUN    ] BufferPoolManagerTest.SimpleTest
[ OK     ] BufferPoolManagerTest.SimpleTest (15 ms)
[ RUN    ] BufferPoolManagerTest.LargeScaleTest
[ OK     ] BufferPoolManagerTest.LargeScaleTest (7849 ms)
[ RUN    ] BufferPoolManagerTest.MultipleFilesTest
[ OK     ] BufferPoolManagerTest.MultipleFilesTest (3446 ms)
[ RUN    ] BufferPoolManagerTest.ConcurrencyTest
[ OK     ] BufferPoolManagerTest.ConcurrencyTest (5912 ms)
[-----] 4 tests from BufferPoolManagerTest (17224 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (17224 ms total)
[ PASSED ] 4 tests.
amdreameng@LAPTOP-34HBN43K:/mnt/d/Desktop/数据库系统/lab/rucbase-lab/build$
```

六、实验结论

实验 1 主要完成了数据库缓冲池层的核心实现：

1. 实现了基于 LRU 的替换器。
2. 实现了页表、空闲链表和替换器三者之间的协同。
3. 实现了页面读取、写回、新建和删除机制。
4. 建立了页的引用计数和脏页管理机制。

该实验为后续记录管理器和执行器提供了最底层的页缓存支持。

实验 2：记录管理器实现

一、实验目标

本实验的目标是掌握 Rucbase 记录管理器的实现方法。

二、实验内容

实验 2 分为两个部分：

1. 完成 RmFileHandle，支持记录的读取、插入、删除和更新。
2. 完成 RmScan，支持对记录文件中所有记录的遍历。

涉及的核心文件如下：

- src/record/rm_defs.h
- src/record/rm_manager.h
- src/record/rm_file_handle.h
- src/record/rm_file_handle.cpp
- src/record/rm_scan.h
- src/record/rm_scan.cpp

从系统层次上看，实验 2 建立的是“记录层”。

实验 1 解决的是“页如何在内存中管理”，而实验 2 则进一步回答：

1. 一条记录在页中怎么存。
2. 如何定位某条记录。
3. 如何判断一个 slot 中是否真的有记录。
4. 如何在整张表中遍历所有记录。

Rucbase 使用的是典型的“页 + bitmap + slot”布局：

1. 一张表对应一个记录文件。
2. 记录文件由多个记录页组成。
3. 每个记录页内部有页头、bitmap 和多个固定大小的 slot。
4. Rid(page_no, slot_no) 唯一标识一条记录位置。

三、主要实现代码与实现说明

1. RmFileHandle 的实现

RmFileHandle 面向单个已打开的记录文件，负责文件内部的记录操作。

(1) 关键结构说明

理解本实验时，需要先把几个核心概念分清：

1. RmRecord
表示一条记录在内存中的副本。
2. Rid
表示记录的逻辑地址，由页号和槽号组成。
3. RmPageHandle
表示一个记录页的结构化视图，包含页头、bitmap、slot 等信息。
4. file_hdr_
表示整个记录文件的头部信息，例如记录长度、每页可存记录数、页数、空闲页链表入口等。

因此，RmFileHandle 的所有操作本质上都是围绕：

1. 找页。
2. 找 slot。
3. 判断 slot 是否有效。
4. 读写 slot 内容。

(2) RmFileHandle::get_record

```
std::unique_ptr<RmRecord> RmFileHandle::get_record(const Rid &rid, Context *context) const {
    if (rid.page_no < RM_FIRST_RECORD_PAGE || rid.page_no >= file_hdr_.num_pages ||
        rid.slot_no < 0 || rid.slot_no >= file_hdr_.num_records_per_page) {
        throw RecordNotFoundError(rid.page_no, rid.slot_no);
    }

    RmPageHandle page_handle = fetch_page_handle(rid.page_no);
    if (!Bitmap::is_set(page_handle.bitmap, rid.slot_no)) {
        buffer_pool_manager->unpin_page(page_handle.page->get_page_id(), false);
        throw RecordNotFoundError(rid.page_no, rid.slot_no);
    }

    auto record = std::make_unique<RmRecord>(file_hdr_.record_size,
        page_handle.get_slot(rid.slot_no));
    buffer_pool_manager->unpin_page(page_handle.page->get_page_id(), false);
    return record;
}
```

实现说明：

get_record() 的执行流程如下：

1. 首先检查 Rid 是否越界：
 - a) page_no 不能小于记录页起始页号。
 - b) page_no 不能超过当前文件页数。
 - c) slot_no 不能超出每页记录槽位数。
2. 调用 fetch_page_handle(page_no) 获取该页。
3. 使用 Bitmap::is_set() 判断该 slot_no 对应位置是否真的有记录。
4. 如果 bitmap 中该位为 0，则说明这个位置虽然逻辑上存在，但当前并没有存放有效记录，应抛出记录不存在异常。
5. 若记录存在，则从对应 slot 复制出长度为 record_size 的一段数据，构造 RmRecord。
6. 最后要记得对该页调用 unpin_page()，释放缓冲池中的引用。

可以看到，这里“记录存在”有两个条件：

1. Rid 位置合法。
2. bitmap 中对应位为 1。

(3) RmFileHandle::insert_record

```
Rid RmFileHandle::insert_record(char *buf, Context *context) {
    RmPageHandle page_handle = create_page_handle();
    int slot_no = Bitmap::first_bit(false, page_handle.bitmap, file_hdr_.num_records_per_page);

    memcpy(page_handle.get_slot(slot_no), buf, file_hdr_.record_size);
    Bitmap::set(page_handle.bitmap, slot_no);
}
```

```

page_handle.page_hdr->num_records++;

if (page_handle.page_hdr->num_records == file_hdr_.num_records_per_page) {
    file_hdr_.first_free_page_no = page_handle.page_hdr->next_free_page_no;
    page_handle.page_hdr->next_free_page_no = RM_NO_PAGE;
}

Rid rid{page_handle.page->get_page_id().page_no, slot_no};
buffer_pool_manager_>unpin_page(page_handle.page->get_page_id(), true);
return rid;
}

```

实现说明：

插入记录的核心难点在于“找一个可以插入的 slot”。

该函数的流程为：

1. 调用 `create_page_handle()` 找到一个未分页，或者在必要时创建一个新页。
 2. 通过 `Bitmap::first_bit(false, ...)` 找到第一个空 slot。
 3. 使用 `memcpy` 将输入缓冲区 `buf` 写入该 slot。
 4. 将 `bitmap` 对应位置设为 1，表示该 slot 已被占用。
 5. 更新页头中的 `num_records`。
 6. 如果该页在插入后变成满页，则要更新文件头中的 `first_free_page_no`，因为这个页不能再继续作为空闲页使用。
 7. 构造新记录对应的 `Rid` 返回。
 8. 调用 `unpin_page(..., true)`，因为本次插入修改了页内容，页应被标记为脏页。
- 这里最关键的设计是“文件头维护空闲页链表”，这样就不必在每次插入时扫描所有页寻找空位。

(4) `RmFileHandle::delete_record`

```

void RmFileHandle::delete_record(const Rid &rid, Context *context) {
    RmPageHandle page_handle = fetch_page_handle(rid.page_no);
    if (!Bitmap::is_set(page_handle.bitmap, rid.slot_no)) {
        buffer_pool_manager_>unpin_page(page_handle.page->get_page_id(), false);
        throw RecordNotFoundError(rid.page_no, rid.slot_no);
    }

    bool was_full = (page_handle.page_hdr->num_records == file_hdr_.num_records_per_page);
    Bitmap::reset(page_handle.bitmap, rid.slot_no);
    page_handle.page_hdr->num_records--;

    if (was_full) {
        release_page_handle(page_handle);
    }

    buffer_pool_manager_>unpin_page(page_handle.page->get_page_id(), true);
}

```

实现说明：

删除记录的步骤如下：

1. 根据 `rid.page_no` 获取对应页。
2. 使用 `bitmap` 检查该 `slot` 是否真的存放记录。
3. 若存在，则将 `bitmap` 中该位清零。
4. 将页头中的记录数减一。
5. 如果该页删除前是满页，那么删除后就会从“满页”变成“未分页”，这时必须调用 `release_page_handle()`，把它重新挂回空闲页链表。
6. 最后释放缓冲池引用，并标记该页为脏页。

这里的关键点在于：

删除不只是“把记录删掉”，还会影响页是否属于空闲页集合。

(5) `RmFileHandle::update_record`

```
void RmFileHandle::update_record(const Rid &rid, char *buf, Context *context) {
    RmPageHandle page_handle = fetch_page_handle(rid.page_no);
    if (!Bitmap::is_set(page_handle.bitmap, rid.slot_no)) {
        buffer_pool_manager->unpin_page(page_handle.page->get_page_id(), false);
        throw RecordNotFoundError(rid.page_no, rid.slot_no);
    }

    memcpy(page_handle.get_slot(rid.slot_no), buf, file_hdr_.record_size);
    buffer_pool_manager->unpin_page(page_handle.page->get_page_id(), true);
}
```

实现说明：

更新操作比插入和删除更直接：

1. 先确认目标记录存在。
2. 将新数据直接覆盖到原有 `slot` 中。
3. 最后把页作为脏页释放回缓冲池。

之所以可以直接覆盖，是因为当前实验的记录布局是定长的，不存在变长字段带来的空间重排问题。

(6) `RmFileHandle::fetch_page_handle`

```
RmPageHandle RmFileHandle::fetch_page_handle(int page_no) const {
    if (page_no < RM_FIRST_RECORD_PAGE || page_no >= file_hdr_.num_pages) {
        throw PageNotExistError("", page_no);
    }

    PageId page_id{fd_, page_no};
    Page *page = buffer_pool_manager->fetch_page(page_id);
    if (page == nullptr || page->get_page_id().page_no == INVALID_PAGE_ID) {
        throw PageNotExistError("", page_no);
    }
    return RmPageHandle(&file_hdr_, page);
}
```

实现说明：

这个函数负责把“逻辑页号”转换为“可操作的记录页对象”。

流程是：

1. 先检查页号是否合法。
2. 调用缓冲池管理器获取该页。
3. 将返回的 Page* 包装成 RmPageHandle。

也就是说，记录层并不直接操作原始页指针，而是通过 RmPageHandle 访问页面中的页头、bitmap 和 slot。

(7) RmFileHandle::create_new_page_handle

```
RmPageHandle RmFileHandle::create_new_page_handle() {
    PageId page_id{fd_, INVALID_PAGE_ID};
    Page *page = buffer_pool_manager_ -> new_page(&page_id);
    if (page == nullptr) {
        throw InternalError("Failed to allocate a new record page");
    }

    file_hdr_.num_pages++;
    file_hdr_.first_free_page_no = page_id.page_no;

    RmPageHandle page_handle(&file_hdr_, page);
    page_handle.page_hdr -> next_free_page_no = RM_NO_PAGE;
    page_handle.page_hdr -> num_records = 0;
    Bitmap::init(page_handle.bitmap, file_hdr_.bitmap_size);
    return page_handle;
}
```

实现说明：

该函数在没有任何可用未满足页时被调用，用来创建一个新的记录页。

其步骤为：

1. 调用缓冲池分配一个新的物理页。
2. 更新文件头中的总页数。
3. 将新页登记为当前的 first_free_page_no。
4. 初始化页头中的记录数和空闲页指针。
5. 调用 Bitmap::init() 初始化位图，使所有 slot 初始时都为空。

(8) RmFileHandle::create_page_handle

```
RmPageHandle RmFileHandle::create_page_handle() {
    if (file_hdr_.first_free_page_no == RM_NO_PAGE) {
        return create_new_page_handle();
    }
    return fetch_page_handle(file_hdr_.first_free_page_no);
}
```

实现说明：

该函数体现了记录层的重要优化：

1. 若当前还有未满足页，则直接使用文件头维护的第一个空闲页。
2. 若当前不存在空闲页，则创建新页。

因此，插入记录的效率依赖于 first_free_page_no 这一入口指针。

(9) RmFileHandle::release_page_handle

```
void RmFileHandle::release_page_handle(RmPageHandle &page_handle) {
```

```

        page_handle.page_hdr->next_free_page_no = file_hdr.first_free_page_no;
        file_hdr.first_free_page_no = page_handle.page->get_page_id().page_no;
    }

```

实现说明：

当某一页从“满页”变成“未分页”时，需要把它重新接入空闲页链表。

本函数采用头插法完成该操作：

1. 让当前页指向旧的空闲页链表头。
2. 再让文件头的 first_free_page_no 指向当前页。

2. RmScan 的实现

RmScan 负责完成整张表的顺序扫描，是后续顺序查询算子的基础。

(1) RmScan::RmScan

```

RmScan::RmScan(const RmFileHandle *file_handle)
    : file_handle_(file_handle), rid_{RM_FIRST_RECORD_PAGE, -1} {
    next();
}

```

实现说明：

初始化时，并不直接把 rid_ 指向第一条记录，而是先设为“第一页之前”的一个虚拟位置：slot_no = -1。

随后立刻调用一次 next()，这样扫描器创建完成后就已经位于第一条有效记录上。

(2) RmScan::next

```

void RmScan::next() {
    while (rid_.page_no != RM_NO_PAGE && rid_.page_no <
file_handle_->file_hdr_.num_pages) {
        RmPageHandle page_handle = file_handle_->fetch_page_handle(rid_.page_no);
        int next_slot = Bitmap::next_bit(true,
                                page_handle.bitmap,

file_handle_->file_hdr_.num_records_per_page,
                                rid_.slot_no);
        file_handle_->buffer_pool_manager_->unpin_page(page_handle.page->get_page_id(),
false);

        if (next_slot < file_handle_->file_hdr_.num_records_per_page) {
            rid_.slot_no = next_slot;
            return;
        }

        rid_.page_no++;
        rid_.slot_no = -1;
    }

    rid_ = Rid{RM_NO_PAGE, -1};
}

```

实现说明：

顺序扫描的本质流程是：

1. 在当前页中利用 bitmap 查找“下一个被占用的 slot”。
2. 若当前页中还有记录，则更新 rid_.slot_no 并返回。
3. 若当前页中没有更多记录，则将 page_no 加一，转入下一页继续找。
4. 每次访问完页后都要 unpin，避免扫描期间长时间占用所有页。
5. 若所有页都已扫描完成，则将 rid_ 设置为 RM_NO_PAGE，表示结束。

该实现的关键点在于：

它不是逐 slot 机械遍历，而是借助 bitmap 快速跳转到下一个有效记录位置。

(3) RmScan::is_end

```
bool RmScan::is_end() const {  
    return rid_.page_no == RM_NO_PAGE;  
}
```

实现说明：

这里使用 RM_NO_PAGE 作为统一结束标记。

只要 rid_.page_no == RM_NO_PAGE，就说明扫描器已经到达文件末尾。

四、实验问题回答

1. RManager、RmFileHandle 和 RmPageHandle 的设计分别是什么？

- RManager 负责记录文件的创建、打开、关闭和删除，是文件级管理器。
- RmFileHandle 面向单个已打开的记录文件，负责文件内部的记录操作。
- RmPageHandle 是记录页的结构化包装，用于访问页头、bitmap 和 slot。

三者之间的关系是：

RManager 先打开文件得到 RmFileHandle，RmFileHandle 再通过 RmPageHandle 组织页内记录布局。

2. Rid 的作用是什么？

Rid 是记录的逻辑地址，由 page_no 和 slot_no 组成。

数据库通过它唯一定位某一条记录。

3. RmFileHandle::get_record 是如何实现的？

先检查 Rid 是否越界，再获取对应页，通过 bitmap 判断 slot 是否有效，最后复制对应记录内容返回。

4. RmFileHandle::insert_record 是如何实现的？

先找到一个未满页，再在 bitmap 中找到空 slot，把记录写入该 slot，并更新页头和文件头中的空闲页信息。

5. RmFileHandle::delete_record 是如何实现的？

通过 bitmap 确认记录存在后，清除对应位并减少页内记录数；如果该页由满变未满，则重新加入空闲页链表。

6. RmFileHandle::update_record 是如何实现的？

先确认记录存在，然后直接覆盖对应 slot 中的数据区域，最后将该页作为脏页释放回缓冲池。

7. RmScan 是如何遍历记录文件的？

它依赖 bitmap 查找每页中的下一个有效 slot，当前页结束后自动跳到下一页，直到 rid_.page_no 变成 RM_NO_PAGE。

五、测试结果

```
amdreameng@LAPTOP-34HBN43K:/mnt/d/Desktop/数据库系统/lab/rucbase-lab/build$ ./bin/record_manager_test
Running main() from /mnt/d/Desktop/数据库系统/lab/rucbase-lab/deps/googletest/googletest/src/gtest_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from RecordManagerTest
[ RUN     ] RecordManagerTest.SimpleTest
insert 432
delete 287
update 281
[ OK      ] RecordManagerTest.SimpleTest (879 ms)
[ RUN     ] RecordManagerTest.MultipleFilesTest
[ OK      ] RecordManagerTest.MultipleFilesTest (1282 ms)
[-----] 2 tests from RecordManagerTest (2162 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (2162 ms total)
[ PASSED ] 2 tests.
```

六、实验结论

实验 2 在实验 1 的页管理基础上，建立了完整的记录层能力：

1. 实现了基于 Rid 的记录定位。
2. 实现了基于 bitmap 的 slot 管理。
3. 实现了记录的读取、插入、删除和更新。
4. 实现了对整张表中所有记录的顺序遍历。

这为后续的查询执行层提供了直接的数据访问接口。

实验 3：数据定义的实现

一、实验目标

本实验的目标是：

1. 掌握 Rucbase 元数据管理的实现方法。
2. 掌握 Rucbase 数据定义语句的实现方法。

二、实验内容

本实验要求补全 SmManager，使数据库能够支持以下语句或命令：

1. CREATE TABLE
2. DROP TABLE
3. show tables
4. desc

涉及的核心文件包括：

- src/system/sm_meta.h
- src/system/sm_manager.h
- src/system/sm_manager.cpp

从系统层次上看，实验 3 建立的是“元数据层”和“DDL 层”。

前两个实验只解决了页和记录如何存储，而实验 3 进一步解决：

1. 数据库里有哪些表。
2. 每张表有哪些列。
3. 每列的数据类型、长度和偏移量是什么。
4. 表文件和索引文件如何被打开和关闭。

只有这部分完成后，数据库才真正具备“结构化表”的概念。

三、主要实现代码与实现说明

1. 元数据结构设计说明

在阅读 sm_meta.h 和 sm_manager.h 时，可以先把几个核心结构的层次理清：

1. ColMeta
表示列的元数据，例如列名、类型、长度、偏移量和是否建立索引。
2. TabMeta
表示一张表的元数据，由多个 ColMeta 组成，并且还保存该表上的索引信息。
3. DbMeta
表示整个数据库的元数据，保存当前数据库中所有表的元数据。

可以简单理解为：

1. DbMeta 管整库。
2. TabMeta 管单表。
3. ColMeta 管单列。

SmManager 则是这套元数据的管理者。

2. SmManager 的核心成员说明

SmManager 中有两个非常重要的成员：

1. fhs_
保存 table name -> RmFileHandle 的映射。

也就是说，它缓存了当前数据库里每张表已经打开的记录文件句柄。

2. ihs_

保存 index name -> IxIndexHandle 的映射。

它缓存了当前数据库里已经打开的索引文件句柄。

这两个成员的作用是把“元数据层”和“物理文件层”连接起来。

元数据告诉系统“有哪些表和索引”，而 fhs_ 与 ihs_ 让系统真正能够访问这些文件。

3. SmManager::open_db

```
void SmManager::open_db(const std::string& db_name) {
    if (!is_dir(db_name)) {
        throw DatabaseNotFoundError(db_name);
    }
    if (chdir(db_name.c_str()) < 0) {
        throw UnixError();
    }
    std::ifstream ifs(DB_META_NAME);
    ifs >> db_;

    for (auto &entry : db_.tabs_) {
        TabMeta tab_meta = entry.second;
        fhs_[tab_meta.name] = rm_manager_ -> open_file(tab_meta.name);

        for (auto &index : tab_meta.indexes) {
            std::string index_name = ix_manager_ -> get_index_name(tab_meta.name,
index.cols);
            ihs_[index_name] = ix_manager_ -> open_index(tab_meta.name, index.cols);
        }
    }
}
```

实现说明：

数据库打开操作并不只是“进入一个目录”，它需要完整恢复数据库运行时状态。

该函数的步骤为：

1. 判断数据库目录是否存在。
2. 切换当前工作目录到该数据库目录。
3. 打开元数据文件 DB_META_NAME，将文件中的数据库结构读入 db_。
4. 遍历 db_.tabs_ 中的所有表元数据，逐张表打开记录文件并登记到 fhs_。
5. 再遍历每张表上的索引元数据，逐个打开索引文件并登记到 ihs_。

因此，open_db() 完成的是“从磁盘元数据恢复内存运行态”的过程。

4. SmManager::close_db

```
void SmManager::close_db() {
    flush_meta();

    for (auto &item : fhs_) {
        const RmFileHandle* file_handle = item.second.get();
        rm_manager_ -> close_file(file_handle);
    }
}
```

```

    }
    for (auto &entry : ihs_) {
        const IxIndexHandle *index_handle = entry.second.get();
        ix_manager_ ->close_index(index_handle);
    }

    db_.name_.clear();
    db_.tabs_.clear();
    fhs_.clear();
    ihs_.clear();

    if (chdir("..") < 0) {
        throw UnixError();
    }
}

```

实现说明：

关闭数据库时，需要同时完成“落盘”和“资源回收”两件事：

1. 先调用 `flush_meta()`，将当前最新的元数据写回磁盘。
2. 逐个关闭所有表文件句柄。
3. 逐个关闭所有索引文件句柄。
4. 清空内存中的数据库名、表元数据和句柄缓存。
5. 返回上级目录。

这样做可以保证数据库的结构修改不会只停留在内存中。

5. SmManager::create_table

```

void SmManager::create_table(const std::string& tab_name,
                             const std::vector<ColDef>& col_defs,
                             Context* context) {
    if (db_.is_table(tab_name)) {
        throw TableExistsError(tab_name);
    }
    int curr_offset = 0;
    TabMeta tab;
    tab.name = tab_name;
    for (auto &col_def : col_defs) {
        ColMeta col = {.tab_name = tab_name,
                       .name = col_def.name,
                       .type = col_def.type,
                       .len = col_def.len,
                       .offset = curr_offset,
                       .index = false};
        curr_offset += col_def.len;
        tab.cols.push_back(col);
    }
    int record_size = curr_offset;
}

```

```

    rm_manager_ ->create_file(tab_name, record_size);
    db_.tabs_[tab_name] = tab;
    fhs_.emplace(tab_name, rm_manager_ ->open_file(tab_name));
    flush_meta();
}

```

实现说明：

建表的核心是“构造一张表的元数据并为其创建物理文件”。

执行步骤可以分为：

1. 判断当前数据库中是否已存在同名表。
2. 遍历输入的列定义 col_defs:
 - 依次生成每一列的 ColMeta。
 - 计算每列在记录中的偏移量 offset。
 - 将所有列元数据加入表元数据 tab.cols。
3. 使用所有列长度之和得到整条记录长度 record_size。
4. 调用记录管理器为该表创建底层记录文件。
5. 将表元数据加入 db_.tabs_。
6. 立即打开表文件句柄并登记到 fhs_。
7. 将新的数据库元数据刷回磁盘。

这里最重要的一点是“偏移量计算”。

因为后续读取一条记录时，系统必须依靠 offset 才能知道某一行从哪一字节开始。

6. SmManager::drop_table

```

void SmManager::drop_table(const std::string& tab_name, Context* context) {
    TabMeta &tab = db_.get_table(tab_name);
    RmFileHandle *file_handle = fhs_[tab_name].get();
    rm_manager_ ->close_file(file_handle);
    rm_manager_ ->destroy_file(tab_name);

    for (auto &index : tab.indexes) {
        std::string index_name = ix_manager_ ->get_index_name(tab_name, index.cols);
        IxIndexHandle *index_handle = ihs_[index_name].get();
        ix_manager_ ->close_index(index_handle);
        ix_manager_ ->destroy_index(index_name, index.cols);
        ihs_.erase(index_name);
    }
    fhs_.erase(tab_name);
    db_.tabs_.erase(tab_name);
}

```

实现说明：

删表操作不只是删除一份元数据，而是要同步删除与该表有关的所有资源：

1. 先从元数据中找到目标表。
2. 关闭表对应的记录文件句柄。
3. 删除表对应的物理记录文件。
4. 遍历该表上的所有索引：
 - 关闭索引文件句柄。

- 删除索引文件。
- 从 `ihs_` 中移除。

5. 最后从 `fhs_` 和 `db_.tabs_` 中删除该表。

这体现了数据库中“表结构”和“物理存储资源”之间的强关联关系。

7. SmManager::show_tables

```
void SmManager::show_tables(Context* context) {
    std::fstream outfile;
    outfile.open("output.txt", std::ios::out | std::ios::app);
    outfile << "| Tables \n";
    RecordPrinter printer(1);
    printer.print_separator(context);
    printer.print_record({"Tables"}, context);
    printer.print_separator(context);
    for (auto &entry : db_.tabs_) {
        auto &tab = entry.second;
        printer.print_record({tab.name}, context);
        outfile << "|" << tab.name << " \n";
    }
    printer.print_separator(context);
    outfile.close();
}
```

实现说明：

该函数的功能是显示当前数据库中的所有表名。

它不仅要把结果输出到客户端，还必须把同样的结果写到 `output.txt`，因为实验测试脚本会通过对比输出文件判断实现是否正确。

四、实验问题回答

1. ColMeta、TabMeta 和 DbMeta 分别表示什么？

- ColMeta 表示单列的元数据，如列名、类型、长度、偏移量等。
- TabMeta 表示单表的元数据，由多个列元数据和索引元数据组成。
- DbMeta 表示整个数据库的元数据，由多张表的元数据组成。

2. SmManager 的作用是什么？

SmManager 是系统管理层的核心组件，负责数据库打开关闭、元数据加载保存、创建表、删除表、创建索引、删除索引以及 `show tables`、`desc` 等数据定义语句的执行。

3. fhs_ 和 ihs_ 分别是什么？

- `fhs_`：保存当前数据库中每张表对应的记录文件句柄。
- `ihs_`：保存当前数据库中每个索引对应的索引文件句柄。

它们用于缓存已经打开的文件资源。

4. open_db 是如何实现的？

通过读取数据库元数据文件恢复 DbMeta，然后打开所有表文件和索引文件，并将句柄登记到 `fhs_` 和 `ihs_` 中。

5. close_db 是如何实现的？

先将元数据写回磁盘，再关闭所有表文件和索引文件，最后清空内存中的数据库状态。

6. drop_table 是如何实现的？

关闭并删除目标表的记录文件，同时删除与该表相关的所有索引文件，并从元数据和句柄缓存中移除对应条目。

五、测试结果

```
i_recvBytes: 1
  Read from client 4:
Waiting for request...
i_recvBytes: 18
  Read from client 4: drop table grade;
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
finish delete database
Unit Test Score: 25.0
amdreameng@LAPTOP-34HBN43K: /mnt/d/Desktop/数据库系统/lab/rucbase-lab/src/test/query$
```

六、实验结论

实验 3 建立了数据库的目录层与数据定义层能力：

1. 实现了数据库元数据的加载和保存。
2. 实现了表的创建与删除。
3. 实现了表文件和索引文件的打开、关闭与维护。
4. 使 Rucbase 真正具备了“表结构”和“数据库目录”的概念。

这为后续查询执行器理解列信息和表信息提供了必要基础。

实验 4：数据操作的实现

一、实验目标

本实验的目标是：

1. 掌握 Rucbase 中火山模型的实现方法。
2. 掌握 Rucbase 数据查询算子的实现方法。
3. 掌握 Rucbase 数据更新算子的实现方法。

二、实验内容

本实验要求完成以下算子：

1. SeqScanExecutor
2. ProjectionExecutor
3. NestedLoopJoinExecutor
4. UpdateExecutor
5. DeleteExecutor

相关文件主要包括：

- src/execution/executor_abstract.h
- src/execution/executor_seq_scan.h
- src/execution/executor_projection.h
- src/execution/executor_nestedloop_join.h
- src/execution/executor_update.h
- src/execution/executor_delete.h
- src/execution/execution_manager.cpp

实验 4 是前面三个实验成果的综合使用。

实验 1 解决了页缓存，实验 2 解决了记录访问，实验 3 提供了元数据。

在此基础上，实验 4 回答了一个更高层的问题：

“给定一条 SQL，数据库如何逐条产生查询结果，或者逐条修改目标记录？”

三、主要实现代码与实现说明

1. 火山模型基础接口

AbstractExecutor 中定义了所有执行器共同遵循的接口：

```
virtual void beginTuple(){};
virtual void nextTuple(){};
virtual bool is_end() const { return true; };
virtual std::unique_ptr<RmRecord> Next() = 0;
```

实现说明：

这些接口分别表示：

1. beginTuple()：将执行器定位到第一条结果。
2. nextTuple()：推进到下一条结果。
3. is_end()：判断是否已经没有结果。
4. Next()：返回当前结果元组。

父执行器通过不断调用子执行器的这组接口，逐条拉取数据并进行处理，这就是 Volcano Model 的本质。

2. 条件判断的公共实现

在 `executor_abstract.h` 中，还实现了非常重要的 `CheckCond()`：

```
bool CheckCond(const RmRecord *l_record, const std::vector<Condition>& conds_, const
std::vector<ColMeta>& cols_) {
    ...
}
```

实现说明：

该函数用于统一判断记录是否满足条件。

它完成的工作包括：

1. 根据 `TabCol` 找到对应列的 `ColMeta`。
2. 根据 `offset` 从记录中取出左值和右值。
3. 调用 `ix_compare()` 进行比较。
4. 根据比较运算符 `CompOp` 判断条件是否成立。

这样一来，顺序扫描、连接、删除等算子都可以复用同一套条件判断逻辑。

3. SeqScanExecutor

顺序扫描算子是最基础的查询执行器。

它的任务是：

1. 扫描一张表中的全部记录。
2. 过滤掉不满足条件的记录。
3. 将满足条件的记录一条条交给上层执行器。

(1) beginTuple

```
void beginTuple() override {
    if (context_ != nullptr && context_ -> lock_mgr_ != nullptr && context_ -> txn_ != nullptr) {
        context_ -> lock_mgr_ -> lock_shared_on_table(context_ -> txn_, fh_ -> GetFd());
    }
    scan_ = std::make_unique<RmScan>(fh_);
    rid_ = scan_ -> rid();

    while (!scan_ -> is_end()) {
        if (CheckCond(fh_ -> get_record(rid_, context_).get(), conds_, cols_)) {
            break;
        }
        scan_ -> next();
        rid_ = scan_ -> rid();
    }
}
```

实现说明：

1. 若有事务上下文，则先给整张表加共享锁。
2. 构造底层记录扫描器 `RmScan`。
3. 将 `rid_` 初始化为当前扫描到的位置。
4. 若当前记录不满足查询条件，则继续向后扫描，直到找到第一条满足条件的记录。

因此，`beginTuple()` 的工作不是简单地“从第一条记录开始”，而是“从第一条满足条件的记录开始”。

(2) nextTuple

```

void nextTuple() override {
    for (scan_ ->next(); !scan_ ->is_end(); scan_ ->next()) {
        rid_ = scan_ ->rid();
        if (CheckCond(fh_ ->get_record(rid_, context_).get(), conds_, cols_)) {
            break;
        }
    }
}

```

实现说明：

该函数负责继续向后查找下一条满足条件的记录。

它的逻辑与 beginTuple() 类似，只是从当前记录之后开始。

(3) Next

```

std::unique_ptr<RmRecord> Next() override { return fh_ ->get_record(rid_, context_); }

```

实现说明：

当前执行器内部并不长期保存整条记录内容，而是只保存 rid_。

当外部真正需要当前记录时，再通过 rid_ 向记录层取出一份 RmRecord。

这种设计的好处是：

1. 推进游标和返回记录相分离。
2. 执行器状态更轻量。

4. ProjectionExecutor

投影算子的作用是“从下层执行器输出的完整记录中，只保留用户需要的列”。

它的工作不是重新扫描表，而是对下层结果做格式重组。

(1) 构造函数

```

ProjectionExecutor(std::unique_ptr<AbstractExecutor> prev, const std::vector<TabCol> &sel_cols)
{
    prev_ = std::move(prev);
    size_t curr_offset = 0;
    auto &prev_cols = prev_ ->cols();
    for (auto &sel_col : sel_cols) {
        auto pos = get_col(prev_cols, sel_col);
        sel_idx_.push_back(pos - prev_cols.begin());
        auto col = *pos;
        col.offset = curr_offset;
        curr_offset += col.len;
        cols_.push_back(col);
    }
    len_ = curr_offset;
}

```

实现说明：

构造投影算子时，需要提前完成两件事情：

1. 确认用户选中了哪些列。
2. 重新计算这些列在投影后结果记录中的偏移量。

例如，下层记录的原始布局可能是：

1. id

2. name
3. major
4. score

但如果用户只查询 name 和 score，投影结果中的偏移量就必须重新从 0 开始计算，而不能继续沿用原始记录布局。

(2) beginTuple 和 nextTuple

```
void beginTuple() override { prev_ ->beginTuple(); }  
void nextTuple() override { prev_ ->nextTuple(); }
```

实现说明：

投影算子本身不负责产生基础记录，因此它对游标的推进完全依赖于执行器。

(3) Next

```
std::unique_ptr<RmRecord> Next() override {  
    auto prev_record = prev_ ->Next();  
    auto rt_record = std::make_unique<RmRecord>(len_);  
  
    for (ColMeta col : cols_) {  
        TabCol tabcol = {col.tab_name, col.name};  
        ColMeta prev_col = *prev_ ->get_col(prev_ ->cols(), tabcol);  
        memcpy(rt_record ->data + col.offset, prev_record ->data + prev_col.offset, col.len);  
    }  
  
    return rt_record;  
}
```

实现说明：

该函数的流程如下：

1. 先向下层执行器取出一条完整记录。
2. 分配一块新的结果记录空间。
3. 遍历所有被选中的列：
 - 找到该列在原始记录中的位置。
 - 找到该列在投影结果中的位置。
 - 将对应字节复制过去。
4. 返回新的投影结果记录。

因此，投影算子的本质是“列抽取和重新布局”。

5. NestedLoopJoinExecutor

嵌套循环连接算子用于完成多表连接。

在本实验中，Rucbase 默认采用嵌套循环连接策略。

(1) beginTuple

```
void beginTuple() override {  
    left_ ->beginTuple();  
    right_ ->beginTuple();  
    isend = left_ ->is_end() || right_ ->is_end();  
    if (isend) {  
        return;  
    }  
}
```

```

        if (CheckCond(get_rec().get(), fed_conds_, cols_)) {
            return;
        }
        advance_to_valid_tuple(true);
    }

```

实现说明：

初始化连接算子时，需要让左右两个子执行器都定位到各自的第一条记录。

如果任意一侧本身为空，则连接结果为空。

若两边都非空，则先检查当前这一对记录组合是否满足连接条件，若不满足，再继续推进。

(2) advance_to_valid_tuple

```

void advance_to_valid_tuple(bool move_left_first) {
    if (isend) {
        return;
    }

    if (move_left_first) {
        left_ ->nextTuple();
    }

    while (!right_ ->is_end()) {
        while (!left_ ->is_end()) {
            if (CheckCond(get_rec().get(), fed_conds_, cols_)) {
                return;
            }
            left_ ->nextTuple();
        }
        right_ ->nextTuple();
        if (!right_ ->is_end()) {
            left_ ->beginTuple();
        }
    }

    isend = true;
}

```

实现说明：

这段代码就是嵌套循环连接的核心：

1. 固定右侧当前记录。
2. 让左侧从头到尾扫描。
3. 对每一对左右记录组合调用 CheckCond() 判断连接条件。
4. 如果左侧扫描结束，就让右侧前进一步，并重新让左侧从头开始。
5. 重复以上过程，直到找到下一条满足条件的连接结果，或者两边都扫描完。

它对应的抽象过程就是两层循环：

1. 外层循环控制右表。
2. 内层循环控制左表。

(3) get_rec

```
std::unique_ptr<RmRecord> get_rec() {  
    std::unique_ptr<RmRecord> record = std::make_unique<RmRecord>(len_);  
    std::unique_ptr<RmRecord> l_rec = left_->Next();  
    std::unique_ptr<RmRecord> r_rec = right_->Next();  
    memset(record->data, 0, record->size);  
    memcpy(record->data, l_rec->data, l_rec->size);  
    memcpy(record->data + l_rec->size, r_rec->data, r_rec->size);  
    return record;  
}
```

实现说明：

连接算子在逻辑上需要把左右两条记录组合成一条更大的结果记录。

因此：

1. 先分配一块足够大的记录空间。
2. 把左记录复制到前半段。
3. 把右记录复制到后半段。

之后无论是条件判断还是上层投影，都可以把它当成一条普通记录来看待。

6. UpdateExecutor

更新算子的作用是对若干目标记录执行 SET 子句指定的修改。

(1) Next

```
std::unique_ptr<RmRecord> Next() override {  
    if (context_ != nullptr && context_->lock_mgr_ != nullptr && context_->txn_ != nullptr) {  
        context_->lock_mgr_->lock_exclusive_on_table(context_->txn_, fh_->GetFd());  
    }  
  
    for (auto &rid : rids_) {  
        auto rec = fh_->get_record(rid, context_);  
        auto old_rec = std::make_unique<RmRecord>(*rec);  
        if (context_ != nullptr && context_->txn_ != nullptr) {  
            context_->txn_->append_write_record(new  
WriteRecord(WType::UPDATE_TUPLE, tab_name_, rid, *old_rec));  
        }  
  
        for (auto &set_clause : set_clauses_) {  
            auto lhs_col = tab_.get_col(set_clause.lhs.col_name);  
            memcpy(rec->data + lhs_col->offset, set_clause.rhs.raw->data, lhs_col->len);  
        }  
  
        fh_->update_record(rid, rec->data, context_);  
    }  
    return nullptr;  
}
```

实现说明：

更新算子的执行流程如下：

1. 如果系统启用了事务和锁管理，则先对整张表加排他锁。
2. 遍历所有目标 rid。
3. 读取旧记录，并复制一份旧版本。
4. 将旧版本写入事务日志，用于后续回滚。
5. 遍历所有 SET 子句：
 - 找到左侧列的元数据。
 - 根据偏移量将新值写入对应列位置。
6. 调用记录层的 `update_record()` 完成真正的覆盖写入。

如果表上建立了索引，则完整实现中还需要同步删除旧索引项并插入新索引项，确保索引和记录内容一致。

7. DeleteExecutor

删除算子的作用是对目标记录执行删除。

(1) Next

```
std::unique_ptr<RmRecord> Next() override {
    if (context_ != nullptr && context_ ->lock_mgr_ != nullptr && context_ ->txn_ != nullptr) {
        context_ ->lock_mgr_ ->lock_exclusive_on_table(context_ ->txn_, fh_ ->GetFd());
    }

    for (Rid rid : rids_) {
        if (!fh_ ->is_record(rid)) {
            continue;
        }

        auto record = fh_ ->get_record(rid, context_);
        if (!CheckCond(record.get(), conds_, cols_)) {
            continue;
        }

        fh_ ->delete_record(rid, context_);
    }
    return nullptr;
}
```

实现说明：

删除算子的执行步骤为：

1. 对表加排他锁。
2. 遍历所有目标 rid。
3. 若某个 rid 已无效，则直接跳过。
4. 重新读取该记录，并再次检查是否满足删除条件。
5. 若满足条件，则调用记录层的 `delete_record()` 完成删除。

如果系统带有事务和索引支持，则完整流程还应包括：

1. 记录删除前的旧值到事务日志。
2. 删除该记录相关的索引项。

8. QlManager::select_from

实验 4 的自动测试为什么能验证查询正确性，还和 `QlManager::select_from` 密切相关：

```
for(executorTreeRoot->beginTuple();!executorTreeRoot->is_end();executorTreeRoot->nextTuple(
)) {
    auto Tuple = executorTreeRoot->Next();
    ...
}
```

实现说明：

这段代码真正驱动整棵执行器树运行：

1. 先让根执行器定位到第一条结果。
2. 每轮取出当前结果元组。
3. 将其格式化输出到客户端。
4. 同时将结果写入 output.txt。
5. 然后推动根执行器前往下一条结果。

因此，实验 4 的测试不只是“函数返回值是否正确”，而是验证整条 SQL 执行链是否正确工作。

四、实验问题回答

1. 什么是火山模型？

火山模型是一种迭代器式查询执行模型。

每个执行器都提供统一的接口，父执行器通过反复调用子执行器的接口逐条拉取元组，从而实现复杂查询的分层执行。

2. SeqScanExecutor 中的 rid_ 有什么作用？

rid_ 表示当前顺序扫描算子定位到的记录位置。

执行器并不永久保存完整记录内容，而是保存逻辑位置，需要时再通过 rid_ 去记录层取出记录。

3. ProjectionExecutor 中的 prev_ 有什么作用？

prev_ 是投影算子的子执行器。

投影算子从 prev_ 获得原始记录，再提取所需列并重组为新的输出记录。

4. NestedLoopJoinExecutor 中的 left_ 和 right_ 有什么作用？

它们分别表示连接算子的左、右子执行器。

连接算子通过不断枚举左右两侧当前元组组合，判断其是否满足连接条件。

5. UpdateExecutor 中的 rids_ 是如何得到的？

rids_ 是前置阶段根据查询条件筛选出的目标记录位置集合。

更新执行器本身负责对这些位置执行修改，而不是重新扫描整表。

6. DeleteExecutor 中的 rids_ 是如何得到的？

与更新算子类似，删除算子的 rids_ 也是在执行前根据条件筛选出的目标记录集合。

7. 为什么实验 4 的测试没有使用 Google Test？

因为实验 4 测试的是完整 SQL 执行链路，而不是单个函数行为。

它需要验证：

1. SQL 被正确解析并构造执行器树。
2. 执行器树可以正确产生结果。
3. 结果能被正确输出到客户端和 output.txt。

因此，它更适合使用 SQL 文件驱动的综合测试，而不是传统的单元测试框架。

五、测试结果


```
Waiting for request...
i_recvBytes: 95
Read from client 4: select id, name, major, course, score from student, grade where student.id = grade.student_id;
Waiting for request...
Maybe the client has closed
Terminating current client_connection...
finish kill
final score: 100.0
amdreameng@LAPTOP-34HBN43K: /mnt/d/Desktop/数据库系统/lab/rucbase-lab/src/test/query$
```

六、实验结论

实验 4 在前面三个实验的基础上，建立了数据库的执行层：

1. 通过火山模型统一了执行器接口。
2. 实现了顺序扫描、投影和嵌套循环连接等查询算子。
3. 实现了更新与删除等数据修改算子。
4. 打通了从 SQL 请求到底层记录操作的执行链路。

完成实验 4 后，Rucbase 已经具备了基础的 SQL 查询与数据更新能力。